

# Building the Stack From Source

This document describes how to build every custom Docker image referenced in `compose.yaml` from source, rather than relying on prebuilt image tarballs. Everything not listed here (Besu, Grafana, Prometheus, MongoDB, LocalStack, the OTEL collector) is a public, unmodified image and is pulled automatically by `docker compose` from its public registry — no build step is needed for those.

There are four custom images. All four are built from the official Madara Alliance v0.10.2 monorepo, with two small, well-understood patches applied to the upstream Dockerfiles, plus one self-contained Node.js gateway that lives directly in this repository.

## 0. Prerequisites

- Docker with BuildKit enabled (default on modern Docker installs)
- At least ~15GB free disk space for build layers and caches
- At least 16GB RAM recommended; Rust compilation of Madara is heavy
- Internet access to clone the Madara monorepo and pull the build-time base images

## 1. Clone the Madara Alliance monorepo at v0.10.2

The three Rust-based images (`madara`, `bootstrapper-v2`, `orchestrator`) are all built from subdirectories of the same upstream monorepo, pinned at the v0.10.2 tag.

```
git clone --branch v0.10.2 https://github.com/madara-alliance/madara.git madara-v0.10.2
cd madara-v0.10.2
```

## 2. Apply the two required patches

Two issues in the upstream v0.10.2 Dockerfiles prevent a clean build as of this writing. Both are environmental (an external URL going stale, and a build artifact that needs to be pre-fetched), not bugs in Madara's logic — but a fresh clone will fail without them.

### Patch A — stale `get-pip.py` URL for Python 3.9

The Python base images used by `bootstrapper-v2/Dockerfile` and `orchestrator/Dockerfile` install pip via `bootstrap.pypa.io/get-pip.py`, which has since dropped support for Python 3.9 and now requires the version-pinned URL.

```
sed -i 's|https://bootstrap.pypa.io/get-pip.py|https://bootstrap.pypa.io/pip/3.9/get-pip.py|' \
    bootstrapper-v2/Dockerfile \
    orchestrator/Dockerfile
```

### Patch B — pre-extract `os_latest.json`

The `bootstrapper-v2` and `orchestrator` builds expect a compiled Cairo OS artifact (`build-artifacts/cairo_lang/os_latest.json`) to already be present in the build context. Upstream publishes this as a separate artifacts image rather than building it from scratch.

```
docker create --name madara-artifacts-extractor ghcr.io/madara-alliance/artifacts:1
docker cp madara-artifacts-extractor:/artifacts.tar.gz ./artifacts.tar.gz
docker rm -f madara-artifacts-extractor
tar -xzf artifacts.tar.gz
ls build-artifacts/cairo_lang/    # should now contain os_latest.json
```

## 3. Build the Madara sequencer/full-node image

This is the largest and slowest build (LLVM 19 + Cairo Native toolchain, full Rust compile of the `madara` binary).

```
docker build -f madara/Dockerfile -t madara:local .
```

Expect this to take a long time on first build (30+ minutes is normal, depending on CPU). Subsequent builds reuse Docker's layer cache as long as the Dockerfile and source tree are unchanged.

## 4. Build the bootstrapper-v2 image

Bootstrapper-v2 deploys both the L1 core contract and L2 system contracts (token bridge, fee token, universal deployer, etc.) at first startup.

```
docker build -f bootstrapper-v2/Dockerfile -t madara-bootstrapper-v2:v0.10.2 .
```

## 5. Build the orchestrator image

The orchestrator drives the proving pipeline (SNOS → PIE → SHARP-compatible prover → L1 settlement). Note it additionally installs Node.js 18 at build time, which the orchestrator's settlement client uses.

```
docker build -f orchestrator/Dockerfile -t madara-orchestrator:v0.10.2 .
```

## 6. Build the stwo-gateway image

Unlike the three images above, this one is not part of the Madara monorepo — it's a custom Node.js service that lives in this repository and implements the SHARP-compatible HTTP API in front of the Stwo prover binary.

```
cd /path/to/AppChain/Layer2/stwo-gateway
docker build -t stwo-gateway:latest .
```

This Dockerfile expects the `stwo-run-and-prove` binary to be available via a bind mount at container runtime (see `compose.yaml`), not baked into the image — so building the Stwo prover binary itself is a separate step.

## 7. Build the Stwo prover binary

The actual proving binary is not a Docker image; it is a Rust binary built directly on the host and bind-mounted into the `stwo-gateway` container.

```
git clone https://github.com/starkware-libs/proving-utils.git proving-utils
cd proving-utils
git checkout 19996f1f80a1b827f283159e37103bc58e20847e
# Use the toolchain pinned in rust-toolchain.toml (nightly-2025-07-14)
cargo +nightly-2025-07-14 update -p ruinet --precise 1.17.0
cargo +nightly-2025-07-14 build --release -p stwo_run_and_prove -j 8
```

The resulting binary will be at `proving-utils/target/release/stwo-run-and-prove`. Place this `proving-utils/` directory at the path referenced by the bind mount in `compose.yaml` (`prover/proving-utils/` relative to the repo root).

## 8. Verify

Once all images are built and tagged as above, and `proving-utils` is built and in place, `compose.yaml` should be able to start the full stack with no further image pulls required for the custom components:

```
./up.sh
```

## Summary of image tags expected by `compose.yaml`

| Image tag                      | Built from                                | Step |
|--------------------------------|-------------------------------------------|------|
| madara:local                   | madara-v0.10.2/madara/Dockerfile          | 1    |
| madara-bootstrapper-v2:v0.10.2 | madara-v0.10.2/bootstrapper-v2/Dockerfile | 2    |
| madara-orchestrator:v0.10.2    | madara-v0.10.2/orchestrator/Dockerfile    | 3    |
| stwo-gateway:latest            | Layer2/stwo-gateway/Dockerfile            | 6    |
| (binary, not an image)         | proving-utils (Stwo, commit               | 7    |
| stwo-run-and-prove             | 19996f1f80a1b827f283159e37103bc58e20847e) |      |

All other services in `compose.yaml` (Besu, Grafana, Prometheus, MongoDB, LocalStack, the OTEL collector, Pathfinder) use unmodified public images and require no build step.